

Sistemas Operativos

Estas son las notas del curso "*Sistemas Operativos*" (SO) de la Licenciatura en Ciencias de la Computación, respectivamente, dictadas por el [Departamento de Computación](#) - [FCEFQyN](#), de la [Universidad Nacional de Río Cuarto](#), Argentina.

Constituyen una guía de los temas estudiados en el curso y contienen los temas fundamentales desarrollados, con referencias bibliográficas de actualidad, manuales técnicos, artículos y a las herramientas de software utilizadas.

Durante el curso desarrollaremos un taller sobre diseño e implementación de sistemas operativos sobre hardware real, basado en la arquitectura RISC-V 32 bits. El proyecto se basa en un *educational operating system* denominado *EDOS* desarrollado por el autor.

En los trabajos prácticos y en los proyectos de taller se utilizan herramientas exclusivamente de software libre.

Autor: [Marcelo Arroyo](#).

Bibliografía

1. Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. University of Wisconsin-Madison. *Operating Systems: Three Easy Pieces*. Free e-book: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
2. Andrew S. Tannenbaum, Herbert Bos. *Modern Operating Systems*. Fifth edition. Pearson. eBook ISBN 13: 978-1-292-72789-9. 2024.
3. Silberschatz, Galvin, Gagne. *Operating Systems Concepts*. 9th Ed. ISBN: 978-0-470-88920-6. 2011.
4. G. Wolf, E. Ruiz, F. Bergero, E. Meza. [Fundamentos de Sistemas Operativos](#). 2015.

5. Prof. Edson Borin. Institute of Computing Unicamp. *An Introduction to Assembly Programming with RISC-V*. Url: <https://riscv-programming.org/book/riscv-book.html>
 6. D. A. Patterson, J. L. Hennessy. *Computer Organization and Design*. RISC-V edition. Morgan Kaufmann publishers. Elsevier. 2018. ISBN: 978-0-12-812275-4.
 7. *Xv6, simple UNIX-like teaching operating system*. MIT.
<https://pdos.csail.mit.edu/6.828/2024/xv6.html>
 8. *RISC-V Specification manuals*:
 - *The RISC-V Instruction Set Manual. Volume I. Unprivileged ISA*
 - *The RISC-V Instruction Set Manual. Volume II. Privileged ISA*
<https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications>
-

Próximo
Introducción

Introducción

Un sistema de computación está formado por múltiples componentes de *hardware* y *software* interconectados. La siguiente figura muestra una estructura típica de componentes en diferentes capas de abstracción en un sistema de computación actual de propósitos generales.

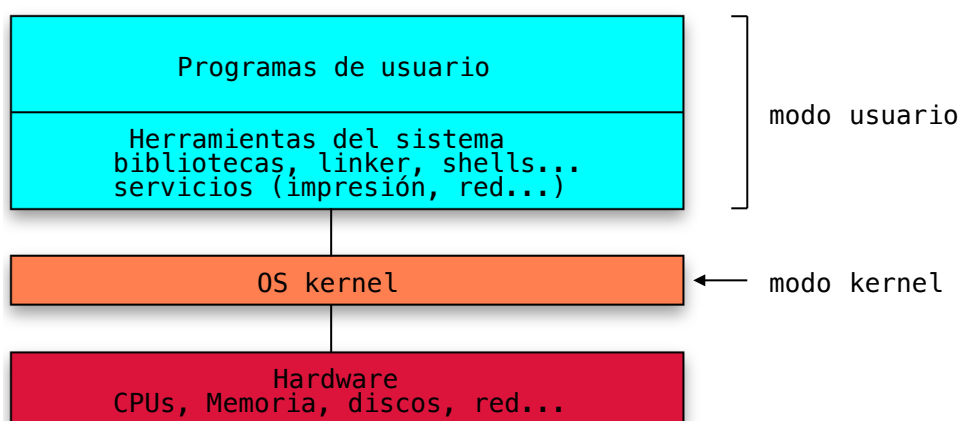


Figura 1: Capas de un sistema de computación.

Uno de los componentes básicos de un sistema de computación es el *sistema operativo (SO)* el cual provee servicios y abstracciones al software de usuarios o aplicaciones.

El desarrollo de un SO presenta desafíos de diseño e implementación por lo que es un tema de investigación desde 1940.

Objetivos de un sistema operativo

Un SO tiene varios objetivos.

1. **Abstraer el hardware:** Por ejemplo, UNIX presenta a los dispositivos como *archivos especiales* y al almacenamiento como un *sistema de archivos*.
2. **Administrar recursos:** Mutiplexa CPUs y dispositivos. Particiona la memoria y provee mecanismos de acceso a recursos.
3. **Proveer servicios:** Las llamadas al sistema proveen servicios de entrada/salida, manejo de procesos, archivos y comunicación.

Algunos sistemas tienen otros objetivos específicos adicionales.

- **Sistemas de escritorio (desktops):** Usados en computadoras personales con entornos gráficos y aplicaciones de escritorio. Ejemplos: MS-Windows, MACOS, GNU-Linux y otros.
- **Para dispositivos móviles:** Soportan interfaces táctiles (*touch screens*) y bajo consumo de energía. Ejemplos: Android, IOS y otros.
- **Sistemas de tiempo real (RTOS):** Usados comúnmente en micro-controladores o *embedded systems*. Garantizan que ciertas operaciones se realicen en un tiempo acotado.

¿Por qué estudiar sistemas operativos?

El estudio del uso, diseño e implementación de SOs permite adquirir habilidades para su análisis y formar partes de equipos que contribuyen al desarrollo de los SO modernos.

Para el programador de aplicaciones, mejora la capacidad de uso de sus servicios y permite la implementación *eficiente* y *segura* de programas.

Muchos de los conceptos desarrollados en el diseño e implementación de SOs, como algunas abstracciones, estrategias y algoritmos de gestión de recursos pueden aplicarse en aplicaciones.

Historia y evolución

En las primeras computadoras (1945-1955), con capacidades de memoria muy limitadas (unas pocas palabras en decimal o binario), cada programa debía contener toda la lógica para controlar los dispositivos de entrada/salida y otras funciones, como cálculos matemáticos de uso común.

La tecnología se basaba en tubos de vacío y mallas de núcleos magnéticos como almacenamiento externo.

Los programas se cargaban en memoria manualmente (*hardwired*) usando conmutadores y patrones de interconexión como se muestra en la siguiente foto.

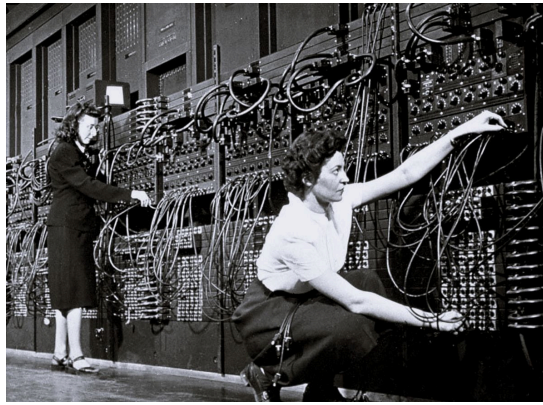


Figura 2: ENIAC, 1946.

Esto era una tarea sumamente tediosa y propensa a errores y sólo era posible hacer pequeños programas para las pequeñas memorias de aquellos días.

Al aumentar la capacidad de memoria, se desarrollaron dispositivos de almacenamiento para luego cargar los programas y datos e imprimir las salidas basados en las siguientes tecnologías:

1. Tarjetas y cintas de papel perforadas
2. Medios magnéticos como las cintas, cilindros y discos
3. Actualmente: Memorias no volátiles de estado sólido

La computadora incluía *lectores* de tarjetas o cintas que introducía las instrucciones y datos de los programas en la memoria y una *impresora* para escribir las salidas.

En cada medio de almacenamiento se usaba una técnica de *codificación* de datos.

Una *teletipo* (*tty*) permitía escribir programas e imprimir cintas o tarjetas perforadas.

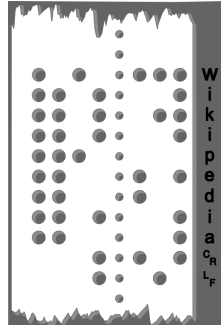


Figura 3: Una *teletipo* con perforadora de cintas.

En esos días no existía el concepto de sistema operativo.

Cada programa debía ser *auto-contenido* incluyendo los subprogramas de entrada-salida y cálculos matemáticos rutinarios. Los programadores debían conocer todos los detalles del sistema.

En las computadoras modernas, su complejidad, cantidad y variedad de dispositivos hace que sea prácticamente imposible que el programador conozca todos los detalles.

Sistemas por lotes (1955-1965)

La tecnología usada se basaba en *transistores*.

Los sistemas denominados *por lotes* o *batch systems* permitían cargar programas y sus dependencias, como el compilador requerido y bibliotecas (*subprogramas*) de uso común.

Los programadores depositaban sus programas y datos de entrada(*jobs*) como una pila de tarjetas perforadas o cintas. Un operador comúnmente las *grababan* en cintas magnéticas para luego poder ser cargados y ejecutados en el sistema.

Cada lote (o *job*) era precedido por una secuencia de comandos que especificaba la secuencia de operaciones a realizar como la carga del compilador, comando de compilación y la carga de las *bibliotecas* requeridas.

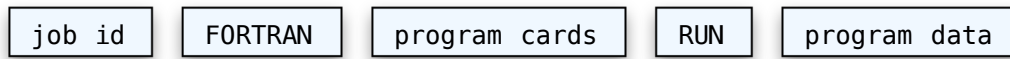


Figura 4: Ejemplo de una secuencia de comandos en un Job.

En estos tipos de sistemas no había mecanismos de *protección*, cada programa tenía acceso a todos los recursos (CPU, memoria, dispositivos de entrada-salida, etc) del sistema.

Circuitos integrados y multiprogramación (1965-1980)

La tecnología de hardware se basa en los circuitos integrados (IC) que contenían un circuito electrónico completo en un *chip*. La capacidad de memoria ya se medía en *kilo bytes (KB)*.

Las operaciones de entrada-salida dejan demasiado tiempo a la CPU ociosa, la cual sólo debía esperar (en un ciclo, verificando algún registro del controlador del dispositivo) a que esas operaciones finalicen.

Con el objetivo de hacer el uso de CPU mas eficiente, los próximos sistemas operativos incluyeron la **multiprogramación** o **multitarea**. El SO puede cargar en memoria varios programas (*procesos*) y ejecutarlos concurrentemente.

Cada tarea al iniciar una operación de E/S solicita un servicio al *kernel* mediante una *llamada al sistema*. Aquí el *kernel* toma el control y puede suspender la tarea actual y asignarle la CPU a otra. Este mecanismo se conoce como *context switch*.

Esta técnica permite *paralelizar* el uso de cpu y los dispositivos de E/S.

Cada tarea tiene un hilo de ejecución y áreas de memoria conteniendo el código (instrucciones de máquina) y datos.

En estos sistemas, las tareas *cooperan* cediendo la CPU en cada llamada al sistema por lo que se conoce también como *multitarea cooperativa*. Las primeras versiones de MS-Windows tenían esta característica.

Protección

La multiprogramación requiere mecanismos de *protección* que se implementan en una combinación de mecanismos de hardware y software.

- **Confinamiento:** Cada proceso debe acceder sólo a su propio *espacio de memoria*.
- **Control:** Los procesos no deberían poder ejecutar ciertas *instrucciones privilegiadas*, como deshabilitar interrupciones o acceder a recursos específicos que permitan quitarle el control al kernel.

En estos sistemas, el *kernel* del SO es un programa independiente y auto-contenido que tiene acceso a todos los recursos del sistema.

A cada programa en ejecución (*proceso* o *tarea*) se asigna su *propio espacio de direcciones de memoria*. Ante un intento de acceso fuera de su espacio, la CPU dispara una *excepción* (o *segmentation fault*).

Los procesos solicitan servicios al SO mediante **llamadas al sistema (syscalls)**.

Definición

Una **Llamada al sistema (syscall)** es una función que permite a los procesos solicitar un servicio o recurso al kernel del SO. Se implementan con instrucciones *trap* o *software interrupts*.

En un *syscall*, el flujo de ejecución del proceso se interrumpe *saltando* del espacio de memoria de un proceso al espacio del kernel, específicamente en un *trap handler* o *interrupt service routine (ISR)*.

Esta rutina invoca al servicio requerido en el kernel y retorna al espacio de usuario del proceso para continuar su ejecución.

Durante el procesamiento de una *syscall* el kernel puede decidir cambiarle de estado al proceso corriente (suspenderlo, por ejemplo) y continuar la ejecución de otro.

Nota

La decisión sobre a qué tarea darle el control está a cargo del *planificador (scheduler)* de procesos.

Con mecanismos de protección, en un salto al kernel, la CPU cambia su *nivel de privilegio* o *modo de operación*. La mayoría de las CPU modernas incluyen al menos dos modos de ejecución: *modo kernel (o supervisor)* y *modo usuario*.

Nota

La CPU ejecuta el código del núcleo del SO en *modo kernel* o *supervisor* (el de mayor privilegio). En **modo kernel** la CPU permite la ejecución de *instrucciones privilegiadas*, acceder a *puertos* de dispositivos, habilitar y deshabilitar interrupciones, entre otras.

Sistemas de tiempo compartido

El hardware evolucionó y se incorporaron dispositivos las *consolas* o *terminales* (teclado + pantalla) se comenzaron a desarrollar aplicaciones *interactivas*

En un sistema con multi-programación una tarea puede usar la CPU por demasiado tiempo y hacer que los usuarios observen demoras excesivas.

Esto motivó el desarrollo de sistemas de **tiempo compartido (timesharing)**.

Definición

Timesharing: Ante una interrupción periódica del reloj, el SO toma el control y puede decidir quitarle la CPU a la tarea corriente y continuar otra.

Esto *multiplexa* la CPU entre las tareas rápidamente dando la sensación a cada usuario que sus procesos progresan sin demoras considerables.

Sistemas multiusuarios

Permiten que varios usuarios inicien *sesión* en el sistema mediante diferentes consolas o terminales.

Los sistemas *multi-usuarios* soportan diferentes espacios de trabajo para diferentes usuarios (*home directories*) con sistemas de *control de acceso* a los recursos. Muchos de estos sistemas permiten la conexión de múltiples *terminales* o *consolas* a la computadora la cual se usa de manera compartida.

Actualmente las computadoras personales permiten *múltiples consolas virtuales*. Una *consola* o *terminal* virtual es un programa que *emula* una terminal física (hardware). También existen aplicaciones que *emulan* terminales remotas por red como `telnet` , `ssh` y aplicaciones de *escritorio remoto* para sesiones en entornos gráficos.

MULTICS y UNIX

En la décadas de 1960 y 1970, MIT, General Electric y Bell Labs desarrollaron un proyecto de investigación y desarrollo de un SO innovador. El resultado fue *MULTiplexed Information and Computing Service* (**MULTICS**) con las siguientes características:

- *Tiempo compartido*
- *Memoria segmentada*: Cada proceso accede a un conjunto de bloques o segmentos.
- *Archivos mapeados en memoria*: La lectura/escritura de archivos se logra mediante accesos a segmentos de memoria.
- *Sistema de archivos jerárquico*: Con directorios y nombres (simbólicos) de archivos.
- *Enlazado dinámico de bibliotecas y servicios (DLLs)*.
- *Interface de línea de comandos*: Incluía un *shell* de comandos, editores de texto y muchas otras utilidades.

En 1969 en Bell Labs Ken Thompson y Dennis Ritchie, decepcionados por la complejidad, tamaño y requerimientos de recursos de MULTICS desarrollaron un SO mas simple con similares características. AT&T lo licenció bajo el nombre de **UNIX**.

El código de versiones previas ya se habían usado por varias empresas y universidades.

Actualmente hay muchas variantes como se muestra abajo.

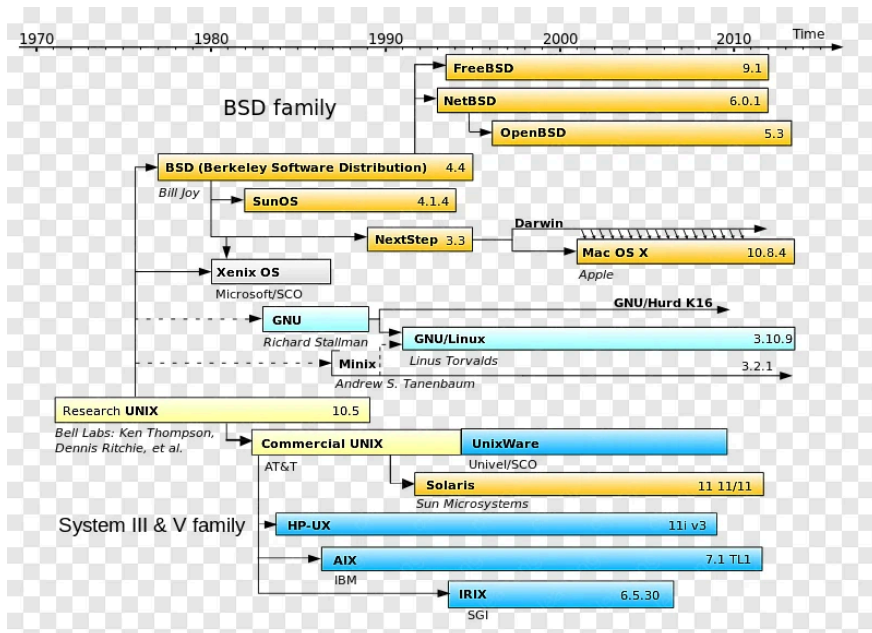


Figura 5: Evolución de UNIX.

UNIX tuvo un gran impacto hasta la actualidad, ya que GNU/Linux se basa en UNIX y es la base de Android y de la mayoría de los servidores y ruteadores de Internet.

Una de sus principales características es su *portabilidad* ya que fue escrito casi enteramente en *C* (lenguaje desarrollado por Dennis Ritchie) con pocas líneas dependientes de la arquitectura en *assembly*.

Hardware

El siguiente diagrama muestra un esquema simplificado de una arquitectura general del hardware.

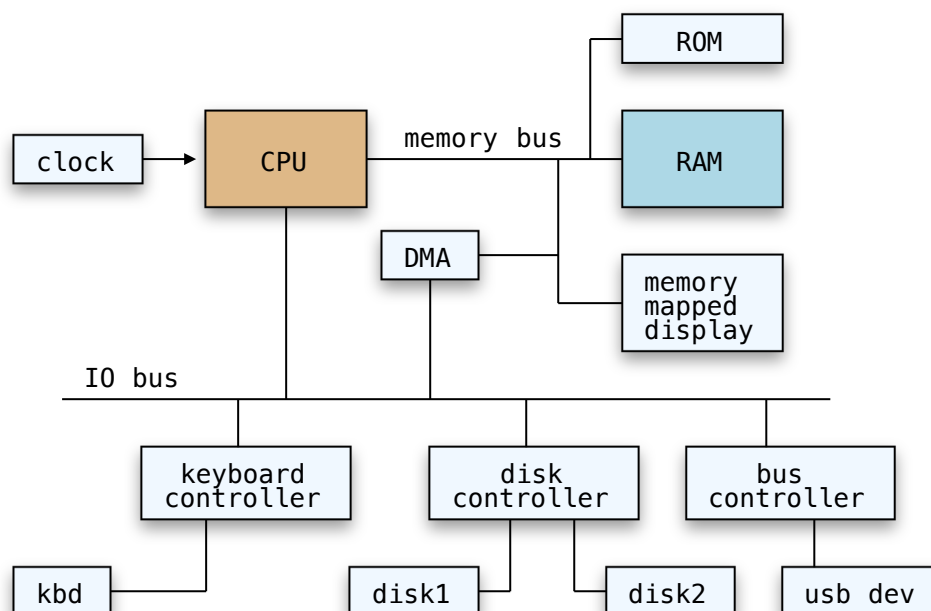


Figura 6: Estructura típica de hardware.

Componentes de hardware:

- *CPU*: Uno o más procesadores.
- *ROM*: Memoria *no volátil* que contiene código de arranque (boot) y datos de la plataforma de hardware.
- *RAM*: Memoria principal (*random-access-memory*). Esta memoria es *volátil*.
- *I/O bus and ports*: Bus de I/O al cual se conectan los dispositivos de entrada/salida como en la plataforma x86 de Intel.
- *Device controller*: expone un conjunto de *puertos* o *registros* de control y estado para un cierto tipo de dispositivos. Un *controller* puede manejar varios dispositivos del mismo tipo. Existen varios estándares como **IDE/PATA** para dispositivos de almacenamiento, **USB** para varios tipos de dispositivos.
- *Memory mapped devices (MMIO)*: Dispositivos de entrada salida conectados al *memory bus*. Ejemplos: *display/video controllers* y otros.
- *Direct-Memory-Access (DMA)*: Dispositivo que inter-conecta controladores de dispositivos al bus de memoria para permitir la transferencia de datos desde/hacia los dispositivos sin necesidad que intervenga la CPU.

Un SO debe controlar y configurar estos componentes. Los módulos de software que interactúan con los dispositivos se denominan *device drivers*.

Diseño

Los SO están diseñados en forma modular, en subsistemas:

1. Núcleo (kernel):

- Gestión de *procesos* y usuarios
- Gestión de la *memoria*
- Subsistema de *entrada-salida*: Conjunto de *device drivers*
- Sistemas de *archivos*
- Subsistema de *red* (comunicaciones)
- Mecanismos de *Seguridad*: Generalmente es un sistema transversal a los demás subsistemas

2. Aplicaciones y *bibliotecas* del sistema (ej: *libc* en sistemas tipo UNIX)

- *Utilidades*: Shells, comandos básicos, editores de texto, ...
- *Servicios*: Procesos de usuario (*daemons*) que brindan servicios de uso común como el servicio de impresión u otros.

Algunos de las procesos de usuario que forman parte de las utilidades y servicios del sistema ejecutan con mayores *privilegios* que los programas de usuario comunes.

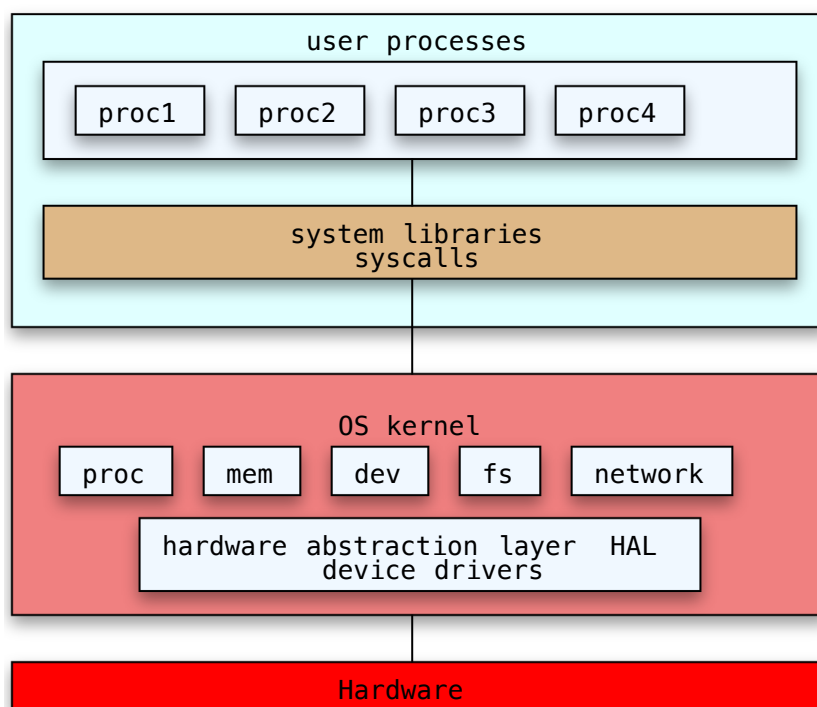


Figura 7: Arquitectura de un sistema de computación.

En la figura 4, los programas de usuario utilizan (*se enlazan con*) las bibliotecas del sistema y otras de propósitos específicas. La biblioteca del sistema contiene la *interfaz del programador de aplicaciones (API)* la cual incluye las funciones que implementan las *llamadas al sistema* y otras utilidades de uso rutinario.

Una *llamada al sistema (syscall)* activa el mecanismo de interrupciones para *saltar* al código del kernel que la procesa.

El subsistema de E/S, representado como *dev* en la figura 2, incluye los *device drivers* los cuales son módulos de software que *controlan* una cierta clase de dispositivos, como discos IDE, teclado, mouse, pantalla, etc.

Uno de los desafíos en el diseño de un SO es encapsular el software dependiente de la plataforma de hardware del resto del código independiente de la plataforma. Esto se logra definiendo abstracciones de los componentes de hardware para facilitar su portabilidad a otras arquitecturas.

Esta capa de abstracción en el *kernel* se conoce como *hardware abstraction layer (HAL)* y se considera la capa más baja dentro del kernel.

Los diseños de los SO se puede clasificar en:

- **Monolíticos:** Los subsistemas del *kernel* se *enlazan* en un solo *ejecutable*. Cada módulo puede *invocar* a funciones de otros módulos.
 - Ventajas: Simple y eficiente. La comunicación entre módulos se realiza mediante simples *invocaciones a funciones*.
 - Desventajas: Un error en un módulo puede afectar a todo el núcleo.

Ejemplos: UNIX, [GNU-Linux][<https://www.gnu.org/gnu/linux-and-gnu.en.html>], [FreeBSD](#) y [Fuchsia](#).

- **Microkernels:** Cada subsistema del *kernel* se compila como un *programa* independiente. Para *invocar* un servicio de otro subsistema se usa un mecanismo de comunicación entre procesos basado en *mensajes*.

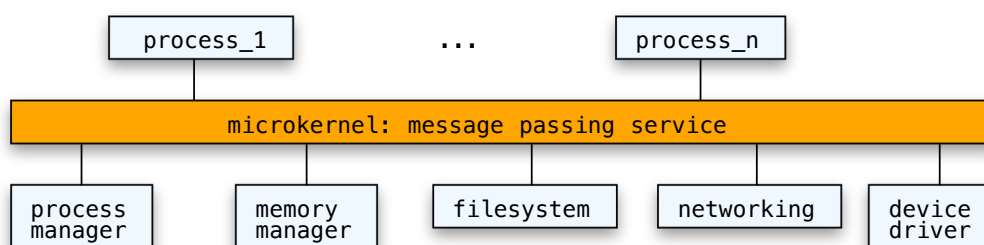


Figura 8: Arquitectura de microkernel.

Cada módulo del SO se implementa como un *servidor* que realiza funciones basados en los mensajes recibidos. Los mensajes básicamente implementan un mecanismo de *remote procedure call (RPC)*.

- Ventajas: Un error (ejemplo, de memoria) en un subsistema no afecta a los demás ya que cada módulo es independiente y está confinado en su propia área de memoria.

Diseño aducado para implementar SOs 'distribuidos' (subsistemas ejecutándose en diferentes nodos de una red).

- Desventajas: El mecanismo de pasaje de mensajes crea una *sobrecarga* computacional importante afectando su rendimiento debido a los múltiples *cambios de contexto* requeridos.

Ejemplos: [Minix](#), [L4](#), [MS-Windows](#).

- **Máquinas virtuales:** El sistema tiene un *hypervisor* que se encarga de *multiplexar* el hardware a los SO *huéspedes* que se ejecutan encima (en modo usuario). El *hypervisor* ofrece a cada SO huésped un *ambiente aislado* de ejecución.

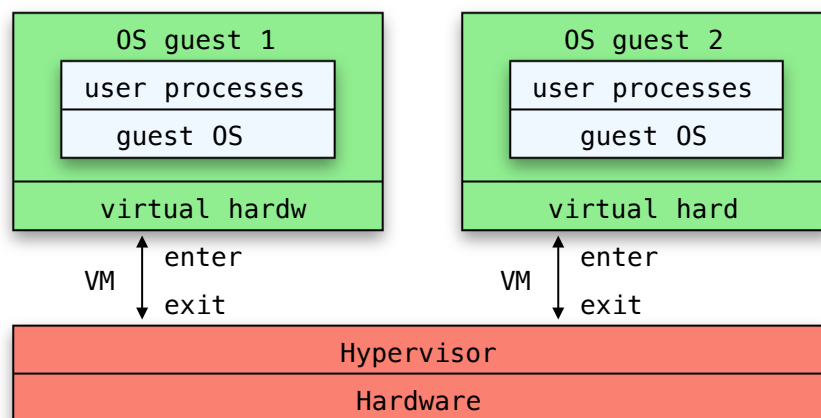


Figura 9: Sistema con máquinas virtuales.

Cuando el *hypervisor* está integrado en un SO, a éste se le denomina el *host kernel (anfitrión)* como por ejemplo [Linux KVM](#).

Generalmente para poder implementar un hypervisor, se usan mecanismos como nuevos *modos de operación* de la CPU como por ejemplo *root mode* (modo del *anfitrión*) y *non-root mode* (modo del *huésped*) y otras estructuras de datos como *registros sombras (shadowed)* por cada OS *huésped*.

Algunas arquitecturas modernas como RISC-V incluyen modos de protección o *privilegios* adicionales para facilitar la implementación de la virtualización.

No se debe confundir la virtualización con *contenedores*. Un contenedor es un *grupo de procesos* y recursos (memoria, filesystem, dispositivos virtuales y otros) que son independientes del resto del sistema que usan el mismo kernel. Esto permite la creación de *subsistemas auto-contenidos* con todas sus dependencias aislados del resto del sistema. Mecanismos: GNU-Linux *cgroups* y FreeBSD *jails*. Aplicaciones como [Docker](#) permiten crear, configurar y ejecutar contenedores.

Consideraciones de diseño

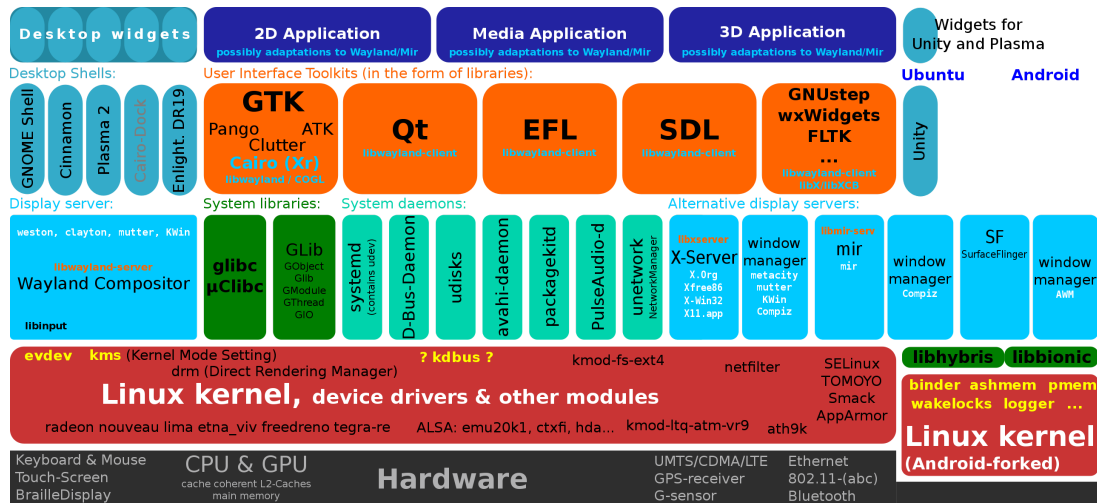
Muchos SO actuales tienen una arquitectura híbrida, generalmente con un kernel pequeño monolítico y otros componentes *fuera de él* comunicándose por mensajes. Ejemplos de estos diseños son MS-Windows y [Minix](#).

Algunos SO modernos, aún cuando sean monolíticos como GNU-Linux, son altamente modulares, permitiendo el desarrollo de ciertos subsistemas como *device drivers* o *filesystems* como *módulos de carga y descarga dinámica* (ver [Linux Kernel Modules](#)), permitiendo al administrador del sistema cargar o descargar componentes *en caliente*, es decir sin necesidad de reiniciar (*reboot*) el sistema.

Otros SO, como [MAC-OS](#) usan un *diseño híbrido*. El kernel, basado en [XNU](#) incluye un conjunto base de funcionalidades como la gestión de tareas, manejo de memoria , controladores de dispositivos críticos, sistemas de archivos y protocolos de comunicación (networking y mensajes). El resto del sistema corre en modo usuario al igual que la mayoría de los controladores de dispositivos.

Un principio de diseño sugerido para los SO es definir e implementar *mecanismos* en las capas mas bajas y que las *políticas* se definan en los niveles mas altos. Esto garantiza una buena *adaptabilidad* de un kernel a diferentes escenarios.

Abajo se muestra los módulos o componentes de GNU/Linux de escritorio.



Otros diseños de SOs son los basados en exokernels

Interfaz de un SO

Un SO presenta a los desarrolladores de aplicaciones interfaces, tanto a nivel de código fuente como en formatos binarios.

Definición

Application Programming Interface (API): Interfaz a nivel de *código fuente* que incluye las llamadas al sistema y las funciones (y tipos de datos) de la biblioteca estándar.

El estándar portable OS interface (POSIX) define una *API* de portabilidad a nivel de sistema y usuario como también con un conjunto de *utilidades* (como *shells* y otros comandos) con el fin de compatibilizar variantes del SO *UNIX*.

En particular, éste estándar define la *API* como declaraciones de funciones y tipos de datos en `c`.

Definición

Application Binary Interface (ABI): Interfaz de bajo nivel que especifica los formatos binarios y convenciones para permitir la interacción de programas de usuario con las *bibliotecas* y con el *kernel* del sistema operativo.

Los sistemas operativos generalmente especifican una *ABI* para cada plataforma de hardware (x86, x86-64, ARM, risc-v, etc). La *ABI* incluye:

1. Convenciones para las *invocaciones a funciones*.
2. Convenciones para los *syscalls*.
3. Formato de archivos binarios (*objetos y ejecutables*)

A modo de ejemplo, GNU-Linux usa binarios en formato **Executable and Linking Format (ELF)** para todas las plataformas soportadas. La convención para las llamadas a funciones en x86 (32 bits) es que los parámetros se *apilan* por el invocante en orden reverso (de derecha a izquierda) y en x86-64 se pasan hasta 6 argumentos en los registros `rdi` , `rsi` , `rdx` , `rcx` , `r8` , `r9` (el resto en la pila en orden reverso). Los valores retornados quedan en el registro `eax/rax` , respectivamente. En ambas arquitecturas se usa las instrucciones `call/system` para una invocación.

En arquitecturas RISC, como RISC-V, los parámetros se pasan en los registros `a0` , `...` , `a7` y el valor retornado queda en `a0` .

Estas convenciones deben ser respetadas por cualquier compilador de la plataforma.

Inicio del sistema (booting)

Al encenderse el hardware éste está configurado para comenzar la ejecución de código desde una dirección de memoria predeterminada que corresponde a una memoria no volátil (ROM, EPROM, etc) que contiene el programa inicial de carga de un SO.

PC BIOS

En el caso de la IBM-PC, basada en x86, el código (conjunto de funciones o *firmware*) en la ROM se conoce como *Basic Input-Output System (BIOS)*. Las PCs modernas contienen un firmware evolucionado conocido como **UEFI**.

El código de la BIOS simplemente carga el contenido del primer bloque de 512 bytes (*master boot record* o *MBR*) del disco primario en la dirección de RAM física `0x7c00` y *salta* (`jmp`) a esa dirección.

El programa cargado deberá cargar en memoria el kernel o un *second stage boot loader* como **GRUB**. GRUB soporta el uso de múltiples OS (Linux, MS-Windows, etc) en diferentes discos o particiones.

El MBR se *graba* durante la instalación o actualización del sistema.

UEFI

UEFI simplifica el proceso de inicio mediante una partición en el disco (rótulada *EFI*) que contiene archivos de configuración y ejecutables.

Un ejecutable puede ser un *boot loader* (como GRUB) o directamente un kernel con soporte UEFI (contiene una cabecera específica).

El firmware carga el archivo de configuración y presenta un menú de opciones de arranque y luego carga y ejecuta el programa correspondiente (boot loader u OS-kernel).

UEFI también soporta *booteo seguro* que se basa en que un kernel esté *firmado* por el fabricante. En éste caso, las claves privadas de cada fabricante deben almacenarse en la memoria *ROM*. Esto complica las actualizaciones frecuentes, práctica común en sistemas Linux.

Otros tipos de *firmware*

Las arquitecturas basadas en **system on chip** comunes en dispositivos móviles y *embedded systems*, que usan CPUs como ARM o RISC-V, comúnmente están configuradas para cargar el kernel desde algún punto específico de algún dispositivo de almacenamiento integrado o externo como una tarjeta de memoria de estado sólido (SD cards).

En éstos sistemas el kernel se carga a partir de una dirección física de memoria predefinida y especificada para cada *board*.

Inicio del SO

El kernel debe estar compilado y enlazado apropiadamente. En su inicio realiza los siguientes pasos:

1. Inicializa subsistemas: CPUs, trap handlers, device drivers, etc.
2. Lanza el primer proceso de usuario preconfigurado.

Referencias

1. Capítulos 1 y 2 del libro *Modern Operating Systems*.

Anterior
Inicio

Próximo
Interfaces

APIs del sistemas operativo

Un sistema operativo ofrece a las aplicaciones servicios que permiten acceder a recursos del sistema y facilidades como mecanismos de comunicación y sincronización entre procesos y otros.

En este capítulo se describe principalmente la API de sistemas tipo UNIX ya que provee interfaces simples y coherentes. La mayoría de los SO modernos se basan en estas APIs.

La API de UNIX se describe como un conjunto de *funciones C* definidas en la *biblioteca estándar* conocida como `libc` . **POSIX** es un estándar que especifica esta API.

A continuación se muestra una lista parcial de llamadas al sistema correspondientes a operaciones sobre procesos y archivos.

Procesos

- `fork()` : Crea un nuevo proceso (hijo) el cual es una copia del proceso invocante.
- `exit(exit_code)` : Finaliza el proceso con código de salida `exit_code` .
- `pid wait(&exit_code)` : Espera (se bloquea) hasta que finalice un proceso hijo. Retorna el *pid* del hijo y en `exit_code` se recibe el código de salida.
- `getpid()` : Retorna el identificador del proceso corriente.
- `getppid()` : Retorna el identificador del proceso padre.
- `kill(pid)` : Envía una señal (terminación) al proceso con id `pid` .
- `sleep(n)` : Espera (se bloquea) por `n` segundos.
- `exec(filename, args)` : Carga el programa `filename` y lo ejecuta. Reemplaza la imagen de código y datos de la memoria del proceso corriente.
- `sbrk(n)` : Asigna `n` bytes de memoria adicionales al proceso corriente.

Archivos

- `open(filename, mode)` : Abre un archivo en el modo (*read, write, ...*) dado.
- `read(fd, buffer, count)` : Lee `count` bytes del archivo en `buffer` .
- `write(fd, buffer, count)` : Escribe `count` bytes de `buffer` al archivo.
- `close(fd)` : Cierra el archivo (libera recurso).
- `dup(fd)` : Duplica (copia) `fd` en el primer descriptor no usado.
- `pipe()` : Crea un *pipe* para comunicación entre procesos.
- `chdir(dirname)` : Cambia de directorio.
- `mkdir(dirname)` : Crea un directorio.
- `mknod(name, major, minor)` : Crea un archivo *especial* (device).
- `fstat(fd)` : Retorna el *estado* y metadatos (size, ...) del archivo.
- `link(f1, f2)` : Crea un *alias* (`f2`) para el archivo `f1` .
- `unlink(filename)` : Borra el archivo.

GNU-Linux provee un conjunto muy amplio de *syscalls*.

Usuarios, sesiones y shell

Generalmente el uso de un sistema multi-usuario se basa en *sesiones*. El software de *terminal* inicia una sesión generalmente con la *autenticación de un usuario* o *login* donde debe presentar sus *credenciales* como su identificador (*username*) y una *contraseña*.

Luego el sistema lanza un *shell*, con el cual el usuario puede *interactuar* con el sistema mediante *comandos*. La sesión se cierra con `Ctrl-D` o el comando `exit` . Luego `init` , actuando como un servicio (o *daemon*) repite el proceso para esa terminal.

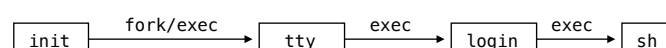


Figura 1: Inicio de una sesión en una terminal.

Por cada usuario del sistema hay una configuración sobre su método de autenticación, el directorio de trabajo (*home*) y el shell a utilizar. En UNIX/Linux, esta información está en `/etc/login`.

El usuario `root` tiene privilegios totales y su rol es de *administrador del sistema*, similar al usuario `admin` o `administrador` en MS-Windows.

Intérprete de comandos (*shell*)

Un *shell* de línea de comandos permite al usuario ingresar comandos para ejecutar programas y soportan operadores sobre el control, sincronización y comunicación entre los procesos creados. En realidad implementan un lenguaje de comandos *Turing computable* o sea que pueden verse como un lenguaje de programación.

Cada SO incluye uno o más *shells* como `bash`, `csh` y `zsh` en sistemas tipo UNIX o `cmd` o `powershell` en MS-Windows.

El shell presenta al usuario un *prompt* configurable (como `$`) indicando que espera un comando.

Como todo intérprete, implementa un ciclo de operaciones **REPL** (*read, execute, print, loop*).

El *directorio* de trabajo del usuario comúnmente es `/home/<username>`. Podrá crear subdirectorios con el comando `mkdir <dirname>` o borrarlos con `rmdir <dirname>` (si está vacío).

El *sistema de archivos* contiene diferentes tipos de componentes:

1. *Directorios*: Contienen archivos y subdirectorios.

Root: Directorio raíz del árbol del sistema de archivos.

Cada subdirectorio tiene dos *directorios especiales*:

1. `.`: Refiere al directorio en sí mismo.
2. `..`: Refiere al directorio padre.

2. *Archivos*:

1. *Programas o archivos ejecutables*: Contienen instrucciones de máquina y datos. Se pueden ejecutar desde el shell o desde otro proceso mediante la llamada al sistema `exec()`.
2. *Datos*: Contienen datos en algún formato (binario o texto).
3. *Archivos especiales*: Representan *dispositivos* como terminales, discos, etc.
4. Otros, como *FIFOs* o *pipes con nombres*

3. *Enlaces (links)*: Referencias a otros archivos/directorios

Un archivo de textos contiene una secuencia de códigos de caracteres (ej: ASCII, UTF-8, ...). Generalmente se estructuran en líneas, es decir, secuencias de caracteres finalizados por `\n` (*newline*) o `\n\r` (*newline/carriage-return*).

Para hacer referencia a un archivo se puede hacer de dos formas:

1. *Full path*: Camino desde la raíz del sistema de archivos hasta el archivo.

ejemplo: `cat /home/user/docs/myfile.txt`

2. *Relativa*: En base al *directorio corriente*, es decir, el directorio de trabajo actual al que se accede mediante el comando `cd dir-path`.

Ejemplo: `cd docs ; ls` (lista el directorio `docs`)

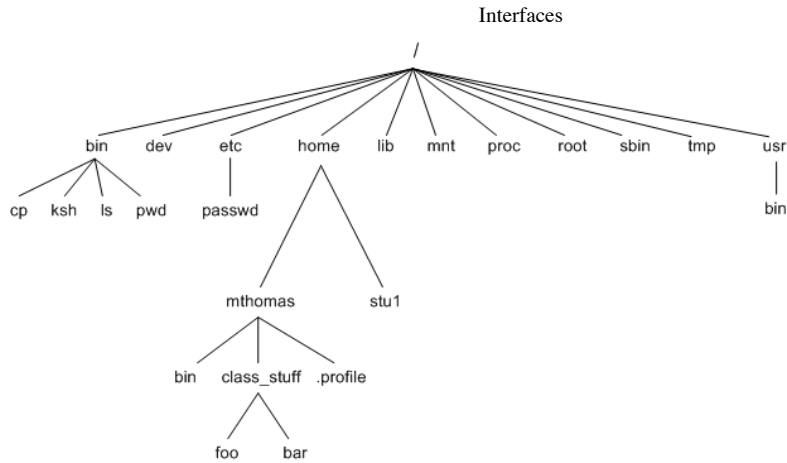


Figura 2: Árbol de directorios en un sistema de archivos UNIX.

Ejecución de comandos

La forma más simple de un comando es

```
$ cmd arg1 arg2 ...
```

Donde `cmd` es el nombre de un *programa* (o su *camino* relativo o full) o un *comando interno* del shell.

El shell también mantiene un conjunto de *variables de ambiente* de la sesión.

En `bash` se definen con el comando `export <VAR>=<value>` . El comando `env` lista las variables de ambiente definidas. Se puede mostrar el valor de una variable usando `echo $<variable-name>` .

```
sh
$ env
SHELL=/bin/bash
USER=marcelo
LANG=es_AR.UTF-8
PATH=/bin:/usr/bin:...
...
```

Los argumentos del comando `arg1`, `arg2`, ... y las variables de ambiente serán pasados al proceso. En programas C se reciben en la función `main` , tal como se muestra en el [Listado]

Si el comando no incluye su *full/relative path*, el shell busca el programa en la lista de directorios dado por la variable de ambiente `$PATH` .

El comando `which <cmd>` permite ver el *full path* del programa correspondiente (si el directorio que lo contiene está en `$PATH`). Ejemplo:

```
sh
$ which cat
/bin/cat
$ _
```

El siguiente programa muestra la API de programación para acceder a los argumentos de la línea de comandos y las variables de ambiente.

El siguiente programa (`myprog.c`) muestra parte de la *API* que nos ofrece un SO *POSIX*.

```
c
#include <sys/types.h> /* system data types */
#include <unistd.h>     /* UNIX standard API */
#include <stdio.h>      /* I/O standard API */

int main(int argc, char *argv[], char *envv[])
```

```

{
    int i;
    printf("Process id: %d\n", getpid()); /* getpid() is a syscall */
    printf("Command line arguments:\n");
    for (i = 0; i < argc; i++)
        printf("%s\n", argv[i]);
    printf("Session environment variables:\n");
    for (i = 0; envv[i]; i++)
        printf("%s\n", envv[i]);
    exit(0); /* exit syscall: process exit code */
}

```

Listado 1: Argumentos de línea de comandos y variables de ambiente.

En este ejemplo se puede observar que:

1. Un programa de usuario comienza por una función denominada `main` que recibe como parámetros el número de los argumentos de la línea de comandos (`argc`), un arreglo de strings (o punteros) `argv` y un arreglo (null-terminated) de strings de la forma `variable=valor` que es la lista de variables de ambiente en la sesión.
2. Los programas interactúan con los dispositivos de entrada/salida (en este caso la *consola* o *terminal*) indirectamente por medio de funciones de biblioteca, las cuales a su vez ejecutan *syscalls*. En este caso `printf()`, internamente ejecuta el *syscall* `write(1, buffer, count)`. El *descriptor de archivo* cero (0) está conectado a la entrada estándar (generalmente el teclado). Los descriptores 1 y 2 refieren a la *salida* y *salida de errores* estándar, asociados a la pantalla.
3. Las llamadas al sistema (como `getpid()` o `exit(n)`) están *implementadas* como funciones de la *biblioteca estándar*.
4. `exit(n)` finaliza el proceso con código de salida *n*. Si en `main()` se omite, se toma el valor retornado por `main` (0 por omisión).

Al compilar este programa (con `gcc -o myprog myprog.c`) el *linker* lo *enlaza* automáticamente con la biblioteca estándar.

Ejecución secuencial de comandos

Es posible *combinar* comandos formando expresiones basados en los siguientes operadores:

- Operador **secuencial**: `cmd1 ; cmd2`

Ejecuta `cmd1` y cuando finaliza, ejecuta `cmd2`.

Cuando el shell ejecuta un comando, el valor de salida de éste (dado por el código de terminación del *syscall* `exit(exit_code)`) queda en la variable de ambiente `$?`.

En los sistemas tipo UNIX la convención es que un proceso con *código de salida=0* significa *terminación normal*, mientras que otros valores representan algún código de error, dependiendo de la aplicación.

Esto permite el uso de *operadores de ejecución secuencial condicional*:

- `cmd1 || cmd2` : Ejecuta `cmd2` si `cmd1` finaliza con error.
- `cmd1 && cmd2` : Si `cmd1` finaliza exitosamente, `cmd2` no se ejecuta.

Estas expresiones pueden asociarse usando paréntesis.



Demo 1: Comandos condicionales.

Operadores de redirección

Cada proceso al iniciarse tiene tres archivos abiertos *heredados desde* `init` (el primer proceso lanzado por el kernel). Cada archivo abierto (mediante el syscall `int open(filepath, mode)`), se representa por su *descriptor*. El número ≥ 0 retornado por `open`.

`init` abre tres archivos (descriptores 0, 1 y 2) asociados a la terminal.

- *Entrada estándar (0)*: Asociado al teclado.
- *Salida estándar (1)*: Salida (*tty display*).
- *Salida de errores estándar (2)*: Salida de mensajes de errores (*tty display*).

Estos tres descriptores de archivos se *heredan* desde `init` en todos los procesos gracias al syscall `fork`.

El shell, al crear un nuevo proceso tiene la oportunidad de *redirigir* esos archivos mediante los *operadores de redirección* `n<` (entrada) y `n>` (salida), donde *n* (opcional) es el número de descriptor de archivo. Si se omite *n* en el caso de entrada es 0 y 1 en el caso de salida.

Syntax: `command > file` o `command < file`

Ejemplos:

1. Crea o sobre-escribe `myfile.txt` con el contenido de la salida estándar (descriptor 1) del comando anterior:

```
$ echo "Hello world" > myfile.txt
```

2. Enviar los mensajes de error del gcc a un archivo:

```
$ gcc myprog.c 2> errors
```

3. El comando `cat` muestra el contenido de un archivo dado como argumento pero puede tomar desde la entrada estándar.

```
$ cat < myfile.txt
```

Control de procesos

Un comando de la forma `cmd &` lanza el programa `cmd` como un *proceso de fondo (background)*. El shell le asigna un *identificador de job* como se muestra abajo.

```
sh
$ (sleep 5 ; echo done) &
[1] 4287
$ _
```

El shell muestra el *job id* (ejecutando en background) 1 asociado al proceso con pid 4287.

Un *proceso en el fondo* si intenta leer o escribir en la terminal se *suspende (stops)*.

`Ctrl-Z` envía al proceso en el frente a *background*.

El comando `fg %job_id` pasa el job al frente.

Es posible *finalizar (matar)* un proceso de fondo mediante el comando `kill -s QUIT %job_id` o `kill -s QUIT pid`.



Demo 2: Control de procesos (background y foreground).

Pipes

Un *pipe* es un mecanismo de comunicación (unidireccional) entre procesos. Un comando de shell de la forma `cmd1 | cmd2` hace que se lancen concurrentemente `cmd1` y `cmd2` previa redirección de la salida estándar al extremo de escritura del *pipe* y la entrada estándar de `cmd2` al extremo de lectura del pipe. En el ejemplo de abajo se muestra un comando que ordena las líneas de texto de los archivos `file1.txt` y `file2.txt` concatenados por `cat`.

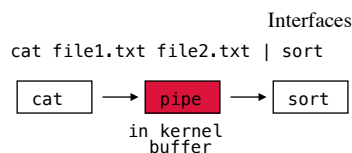


Figura 3: Comunicación entre procesos mediante un UNIX *pipe*.

El *pipe* internamente es un *buffer acotado* asignado por el kernel y éste implementa el clásico patrón *productor/consumidor*. El proceso que escribe puede bloquearse porque el buffer se llenó y el lector se bloquea si el buffer está vacío.

FIFOs: Pipes con nombres

Se debe notar que un *pipe* sólo puede utilizarse por procesos relacionados padre/hijo ya que sólo pueden referenciarse por sus descriptores de archivo.

Un **FIFO** es un pipe al cual se le asigna un nombre en el sistema de archivos. Se pueden crear con el comando `mkfifo <name>`.

Dos procesos independientes pueden comunicarse por un *FIFO* usando las llamadas al sistema `open`, `read`, `write` y `close`.

Cabe aclarar que los datos escritos no se almacenan en el archivo, sino en el *pipe* asociado internamente en el kernel. Cuando los procesos lo cierran el buffer del pipe en el kernel se elimina (libera la memoria).

Interfaces de programación

En esta sección se analizan las llamadas al sistema para la creación, destrucción, sincronización, comunicaciones entre procesos (*Inter Process Communication - IPC*) y manejo de archivos.

Procesos

Un kernel representa los *procesos* o *tareas* como una lista de objetos llamados *process control block (PCB)*.

Definición

Un **proceso** o **tarea** es una *instancia de un programa en ejecución*. En un sistema *multitarea* puede haber varias instancias en ejecución de un mismo programa.

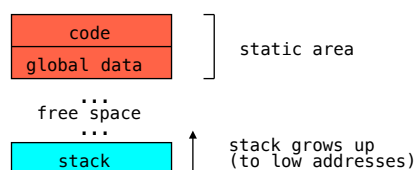


Figura 4: Imagen de un proceso en memoria.

Las propiedades de un *proceso* comúnmente son:

- Su *identificador* o *process id (pid)*. En UNIX es un número entero.
- Su *estado*: `RUNNING`, `READY`, `WAITING`, ...
- El *path* del comando o programa.
- La lista de áreas de memoria en uso.
- Los archivos en uso (abiertos).

De aquí en adelante, analizaremos algunas llamadas al sistema para el control de procesos en sistemas tipo UNIX.

Las principales llamadas al sistema para el manejo de proceso son:

- `fork()` : Crea una copia (un *hijo*) del proceso invocante (el *padre*).

- `exec(path, args)` : Reemplaza la imagen en memoria (código, datos y stack) por el código y datos del programa dado en `path` con una nueva pila con los argumentos y línea de comandos. El proceso pasa a ejecutar el código del programa (comúnmente iniciando por `main()`).
- `exit(code)` : Finaliza el proceso invocante; `code` indica su *código o estado de terminación*.
- `wait(&exit_code)` : Espera (se bloquea) hasta la terminación de algún proceso hijo. Retorna el `pid` del proceso hijo. En `exit_code` queda el código de terminación.

La copia realizada por `fork` implica que el kernel crea una copia en un nuevo descriptor del nuevo proceso y crea copias de las áreas de memoria que contienen el código, datos globales y la pila.

Esto tiene como consecuencia que el hijo *hereda* los archivos abiertos del padre.

Esto hace que cuando el *planificador de procesos* decida otorgarle la CPU al proceso hijo éste iniciará su ejecución en el punto del retorno del `fork()` ya que ese era el contenido del program counter.

En el proceso padre `fork()` retorna el `pid (> 0)` del nuevo proceso (*hijo*). En el hijo retorna 0 (cero).

El siguiente ejemplo muestra el uso de `fork()` .

```
#include <unistd.h> /* UNIX syscalls */
#include <stdio.h> /* portable i/o functions */

int main(void) {
    int pid = fork();
    if (pid == 0) {
        printf("I'm the child. Pid: %d\n", getpid());
    } else {
        printf("I'm the parent. Pid: %d\n", getpid());
        printf("Child pid is: %d\n", pid);
        // wait for child termination
        wait(NULL);
        printf("Child %d terminated.\n", pid);
    }
}
```

Listado 2: Ejemplo de creación de procesos.

Un proceso *padre* debería esperar hasta que sus hijos terminen antes de finalizar.

A continuación se muestra un ejemplo del uso de `wait` y `exit` .

```
#include <unistd.h> /* UNIX API (syscalls) */
#include <stdio.h> /* portable i/o functions */

int main(void) {
    if (fork() == 0) {
        printf("I'm the child. Pid: %d\n", getpid());
        exit(0);
    } else {
        int child, status;
        printf("I'm the parent. Pid: %d\n", getpid());
        child = wait(&status);
        printf("In parent: child %d finished. Exit code: %d\n", child, status);
    }
}
```

Listado 3: Uso de `exit()`/`wait()`.

La llamada al sistema `exec(program, args)` y sus variantes (`execv` , `execve` , ...) reemplaza la imagen de memoria (código y datos) del proceso corriente con el código y datos de `program` .

Cabe aclarar que `exec()` no crea un nuevo proceso. El proceso continúa su ejecución desde el `entry point` especificado en el archivo ejecutable `program`, comúnmente `main()`.

Es común que un programa *lance* otro programa para realizar alguna tarea para luego continuar otras operaciones. En este modelo se debe hacer siguiendo el patrón mostrado en el siguiente ejemplo, el cual ejecuta el comando `ls -l`:

```
#include <unistd.h> /* UNIX syscalls */
#include <stdlib.h> /* wait() */
#include <stdio.h> /* portable i/o functions */

int main(void) {
    char *args[] = {"ls", "-l", 0};

    if (fork() == 0) {
        /* in child */
        execv("/bin/ls", args, 0);
        /* on success, below is unreachable code */
        printf("Ooops, exec() failed!\n");
        exit(-1);
    } else {
        wait(NULL);
        printf("In parent: child finished\n");
    }
}
```

Si compilamos y ejecutamos este programa obtendremos la salida de `ls -l`.



Demo 3: Compilación y ejecución del programa con `fork/exec`.

[!WARINIG|label:Atención] El arreglo de argumentos debe contener el nombre del programa como primer elemento.

Pipes

La llamada al sistema `pipe(p)` retorna en el argumento de salida `p` dos *file descriptors* (enteros). El primer elemento del arreglo corresponde al extremo de lectura del pipe (que usará `cmd2`) y el segundo al de escritura (usado por `cmd1`). En el siguiente ejemplo, el proceso padre envía un mensaje al hijo por un *pipe*.

```
#include <unistd.h>    // UNIX syscalls
#include <stdio.h>      // standard input/output
#include <sys/wait.h>   // wait() syscall

#define N 20

int main(void) {
    int p[2];
    pipe(p);           // create pipe

    if (fork() == 0) {
        // in child process: read from pipe
        char buffer[N];
        close(p[1]);
        // read from pipe
        read(p[0], buffer, N);
        printf("Child read: %s", buffer);
    } else {
        // in parent process: write to pipe
        close(p[0]);
        write(p[1], "Hello from parent\n", 18);
    }
}
```

Listado 4: Comunicación entre procesos por un pipe.

Archivos

Es posible ver los *archivos abiertos de un proceso* como una tabla que el kernel mantiene internamente en el descriptor del proceso como un arreglo de punteros a objetos que tienen los *metadatos* (path, tamaño, tipo, etc) de los archivos.

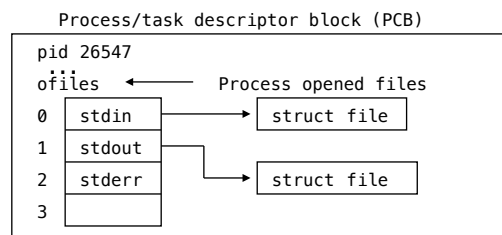


Figura 5: Archivos abiertos de un proceso en el *PCB*.

Para la mayoría de los SOs, un archivo es simplemente una *secuencia de bytes* en un dispositivo de almacenamiento.

Operaciones sobre archivos:

- `int fd = open(path, mode)` : abre el archivo (crea un nuevo elemento en `ofiles`). Retorna el índice del slot libre en `ofiles` usado.

`mode` puede ser:

- `O_CREATE` : Crea el archivo. Si existía, lo recrea vacío.
- `O_RDONLY` : Abre sólo para lectura.

- `O_WRONLY` : Abre sólo para escritura.
- `O_RDWR` : Abre para lectura y escritura.
- `int n = read(fd, buffer, count)` : Lee en el parámetro de salida `buffer` hasta `count` bytes (`count ≤ length(buffer)`). Retorna la cantidad de bytes leídos. Si retorna 0, se ha alcanzado el fin del archivo (*EOF*).
- `int n = write(fd, buffer, count)` : Escribe hasta `count` bytes desde `buffer` al archivo.
- `close(fd)` : Cierra el archivo.

Nota

El kernel mantiene internamente la posición del *cursor* u *offset* de lectura/escritura. Cada operación `read/write` avanza el *offset* en la cantidad de bytes transferidos. El syscall `open` inicia *offset* = 0. Es posible cambiar *offset* mediante `lseek(fd, offset, whence)` .

La biblioteca estándar provee funciones de manejo de archivos independientes de la plataforma. El archivo de cabecera `stdio.h` define sus perfiles.

Redirección

El modelo de ejecución de programas de dos pasos `fork()/exec()` permite al proceso padre *modificar el ambiente de ejecución* del nuevo programa a ejecutar. En particular puede *redireccionar* las entradas y salidas (usando el syscall `dup`).

El siguiente ejemplo permite muestra cómo es posible ejecutar el programa `ls -l` en un proceso hijo para que su salida se envíe al archivo `/tmp/out.txt` .

```
#include <unistd.h>    // UNIX syscalls
#include <stdio.h>      // standard input/output
#include <fcntl.h>      // file operations and flags
#include <sys/wait.h>   // wait() syscall

int main(void) {
    if (fork() == 0) {
        // in child: open/create file with rw permissions
        int fd = open("output.txt", O_WRONLY | O_CREAT, 0600);
        char* args[] = {"ls", "-l", 0};
        printf("File fd: %d\n", fd);    // fd should have the value
        close(1);                      // close stdout
        dup(fd);                       // duplicate fd: ofiles[1] == fd
        // output of "ls -l" goes to output.txt
        execv("/bin/ls", args);
    } else {
        /* in parent */
        wait(NULL);
    }
}
```

Listado 5: Redirección de la salida estándar del proceso hijo.

Nota

La llamada al sistema `dup(fd)` *copia (duplica)* el descriptor `fd` en el primer *slot* libre (cerrado) en el arreglo `ofiles` .

Señales

Las señales son un mecanismo básico de *comunicación entre procesos* para implementar la notificación de *eventos*. El kernel en algunos casos abstraer una *interrupción* o *excepción* en una *señal* enviada a un proceso. También un proceso puede enviar una *señal* a otro proceso. Este mecanismo fue desarrollado en las primeras versiones de UNIX en la década de 1970.

La llamada al sistema `kill(pid, signal)` envía la señal `signal` al proceso identificado por `pid`. Cada tipo de señal es un número (ver con `man signal`).

El shell envía señales a los procesos cuando el usuario en una terminal pulsa las combinaciones de teclas como por ejemplo `Ctrl-C`, la cual envía `SIGINT` (*interrupt*) o `Ctrl-Z`, que envía `SIGSTOP`.

Un proceso puede *manejar* una señal *s* instalando un *signal handler* por medio del syscall `signal(sig, handler)`, donde `sig` es la señal a manejar y `handler` es una función de la forma `void handler(int signal)`.

En el caso que un proceso no tenga definido un *handler* para un tipo de señal, en el caso que ocurra el kernel realizará una acción, dependiendo del tipo de señal. Para la mayoría de las señales el kernel termina el proceso. Para `SIGSTOP` lo suspende.

Cuando el kernel procesa debe continuar la ejecución de un proceso y éste tiene una *señal pendiente*, configura su estado para que al retorno se altere su flujo de continuación normal y el proceso *salte* al handler.

El proceso ejecutará el handler y luego continúa en su flujo de ejecución normal.

La señal `SIGKILL` no puede manejarse, ya que se usa para *terminar* un proceso (si es que el proceso emisor tiene los permisos).

Alarmas

Las llamadas al sistema `alarm(seconds)` y `setitimer(...)` permiten definir una alarma, es decir que luego de transcurrido ese tiempo, el kernel lanzará la señal `SIGALRM` al proceso.

Una alarma generalmente se usa para definir tareas periódicas o implementar *timeouts* de operaciones como en el siguiente ejemplo:

```
#include <signal.h>
#include <unistd.h>
#include <stdio.h>

// signal handler
void on_alarm(int sn)
{
    printf("Please hurry!\n");
    // schedule next alarm
    alarm(5);
}

int main()
{
    char wish[81];

    signal(SIGALRM, on_alarm);
    printf("Enter your wish before next 5 seconds.\n");
    alarm(5);
    scanf("%s", wish);
    // uninstall signal handler.
    signal(SIGALRM, SIG_IGN);
    printf("Your wish is: %s\n", wish);
    return 0;
}
```

Listado 6: Ejemplo de implementación de *timeouts*.

Trazado de procesos

Con llamada al sistema `ptrace(request, pid, addr, data)` un proceso puede monitorear y controlar la ejecución de otro proceso (si tiene los permisos necesarios). Al primero se lo conoce como el *trazador* y al otro el *trazado*. Comúnmente un *debugger* se implementa usando este mecanismo.

El proceso *trazado* se suspende cada vez reciba una señal o haga una llamada al sistema. El *trazador* será notificado en un `wait(&status)` o al recibir la señal `SIGCHLD`. En `status` se indica la causa de la suspensión del proceso trazado.

Luego, el *trazador* puede realizar (mediante otras llamadas a `ptrace()`) alguna de las siguientes acciones sobre el proceso *trazado*:

- Identificar qué llamada al sistema realizó.
- Leer o escribir en el área de código o datos. Así un debugger, puede por ejemplo, definir un *breakpoint* por software, reemplazando una instrucción por una llamada al sistema.
- Acceder al estado de la CPU (valores de los registros) del proceso trazado.
- Continuar la ejecución.
- Otras. Para más detalles, hacer `man ptrace`.

A continuación se muestra un ejemplo simple que *intercepta* la primera llamada al sistema hecha por el proceso hijo (en GNU-Linux arquitectura x86_64).

```
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <sys/user.h>
#include <sys/reg.h>
#include <sys/syscall.h>
#include <stdio.h>

int main()
{
    pid_t child;
    long rax;
    int status;
    child = fork();
    if(child == 0) {
        // child process enables tracing
        ptrace(PTRACE_TRACEME, 0, NULL, NULL);
        execl("/bin/ls", "ls", NULL);
    } else {
        while (1) {
            // the parent is the tracer
            wait(&status);
            // finish tracer if child did an exit syscall
            if (WIFEXITED(status))
                break;
            // get saved rax register value of child process (syscall number)
            rax = ptrace(PTRACE_PEEKUSER, child, 8 * ORIG_RAX, NULL);
            printf("Child syscall number %ld\n", orig_rax);
            // continue tracing next syscall
            ptrace(PTRACE_SYSCALL, child, NULL, NULL);
        }
    }
    return 0;
}
```

En <https://filippo.io/linux-syscall-table/> está el listado de los *syscalls* de Linux en la arquitectura x86-64.

En Linux, el comando `strace` permite *rastrear* un proceso.

Otras APIs

En éste capítulo se analizaron un subconjunto de llamadas al sistema, principalmente aquellas relacionadas al manejo de procesos archivo e IPC básica como pipes y señales.

En los capítulos de manejo de memoria, sistemas de archivos y protocolos de comunicación remotos se analizarán otras APIs expuestas por un SO.

Referencias

1. UNIX tutorial: <https://info-ee.surrey.ac.uk/Teaching/Unix/>

Anterior

Introducción

Próximo

Herramientas de desarrollo